Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 233 (2024) 381–390

Procedia Computer Science

www.elsevier.com/locate/procedia

5th International Conference on Innovative Data Communication Technologies and Application (ICIDCA 2024)

From Posts to Personas: Classifying Users in the Social Media Landscape

Lekshmi S Nair[a,*], Jo Cheriyan[b]

[a]Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, India
[b]Department of Computer Science and Engineering, Saintgits College of Engineering, Kottayam, Kerala, India

lekshmisn@am.amrita.edu

Abstract

Social media is a digital environment where users openly share their opinions and engage in debates and discussions on various topics. Social media has amassed an enormous quantity of accessible data due to its constantly expanding and highly active user base. This data is a valuable resource for researching a variety of topics. An example of such a research problem is identifying user similarities by analyzing their data. This article explores combining comments, textual posts, and likes for comments to create keywords or tags and group the users. These tags are extracted from user data and utilized in constructing a tag network. The tag network facilitates the formation of communities consisting of users with similar interests. User grouping is achieved based on the tags extracted from the posts. The proposed methods employ TF-IDF (term frequency-inverse document frequency) and TextRank algorithms to extract the tags. Kernel diffusion determines the similarities between tags in the tag network. Finally, an aggregation-hierarchical clustering algorithm is employed to group social media users based on these tags.

© 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application

Keywords: Clustering; Complex Networks; Pagerank; Posts; Social Media; Tag Network;

1. Introduction

Social media data lets users express diverse opinions and understand perspectives on specific topics, including text-based posts, comments, likes, and shares. The comments section offers valuable user sentiment insights while liking a post indicates similarities between users' views [1]. Popular social networks YouTube, Facebook, and Instagram provide platforms for creating, commenting, and liking content, facilitating information sharing and open interaction. Users are categorized as content producers and consumers. The article explores user activities to find communities based on their commenting and liking behavior [2], [3].

1877-0509 © 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application
10.1016/j.procs.2024.03.228

Grouping like-minded people has numerous applications and benefits. It fosters socialization, where people form groups based on shared interests, leading to meaningful conversations and shared experiences [4]. In professional spaces, people within the same field network and collaborate. Similarly, people with common hobbies form groups to share activities. Grouping users by preferences is valuable for targeted marketing based on demographics, interests, and buying behavior [5], [6].

The proposed approach focuses on using tags to compare user content efficiently. Tags are precise keywords enabling easy material discovery based on shared interests. User-generated tags are helpful for grouping and filtering social media posts, increasing post visibility and engagement. Tag extraction is valuable for social media platforms, where hashtags group related content. Proper use of relevant hashtags boosts engagement [7].

The approach also recommends user connections based on posting behavior, not link information. The article outlines effective tag extraction algorithms (TF-IDF, TextRank) and user grouping using Agglomerative Hierarchical Clustering [8], [9]. The study's dataset come from Twitter and Facebook, with their unique features for content sharing and reactions.

The rest of the article is organized as : all the relevant state of the art literature and background study is shown in the Section 2. The proposed clustering algorithm, creating tag, and its number of occurrence, the ranking schemes how Agglomerative Hierarchical clustering applied in the proposed methodology are explained in Section 3. In Section 4, the validation procedure, dataset and corresponding result are explained. Finally, the article conclude in Section 5 and proposed suggestion for the future advancements.

2. Related Work

Creating hashtags for social media content is crucial in enhancing post visibility and helping users discover content that matches their interests. Numerous methods have been suggested for hashtag generation and grouping users based on shared interests. Natural Language Processing (NLP) is essential for recognizing possible hashtags by identifying pertinent keywords and phrases. User segmentation is achieved by leveraging NLP for text analysis, image analysis for photos, and video analysis for videos. Social network analysis is utilized to identify users who regularly use similar hashtags, interact frequently, or follow comparable profiles, streamlining the formation of interest-based user communities [10], [11]. For example, hierarchical clustering [12], co-occurrences of words with TF-IDF rankings [13], and user profile-based methods have been utilized for hashtag recommendation.

The algorithm called HF-IHU, similar to TF-IDF, recommends hashtags for new posts based on hashtag frequency and ubiquity. It relies on hashtag popularity and tweet similarity to perform well compared to K-Nearest Neighbors [14]. Another approach incorporates semantic features from word embeddings and user influence features based on influential positions. External sources like Wikipedia are used to overcome limited-word co-occurrence and data sparsity in short tweets. Lexical, topical, and semantic features [15] are combined for hashtag recommendation, utilizing TF-IDF, cosine similarity, and LDA. In terms of user grouping [16], a concept called Tag network is introduced to represent tags and their relationships in a knowledge graph. The approach is compared to Latent Semantic Indexing.

The study also explores finding information spreaders on Twitter, understanding information diffusion, and grouping users based on link information. A study on large-scale social bookmarking websites discusses using user-generated tags to automate web page clustering [17],[18],[19], [20]. The study proposes a generative clustering algorithm based on latent Dirichlet allocation to model text and tags together.Additionally, the article discusses various methods for clustering like-minded people on Twitter, such as K-means, LDA, and Scalable Multi-stage Clustering algorithm (SMSC). Novel approaches called GLIC and GLUCA utilize PCA to group users based on interest centers and concentration axes, employing SVM for classification [21].

The proposed work involves pre-processing text data, extracting keywords using TF-IDF and TextRank, constructing a tag network, measuring tag similarity using kernel diffusion, and then clustering users based on similarity measures using Agglomerative Hierarchical clustering.

3. Proposed Work

During preprocessing, we remove non-English content and HTTP links. Based on TF-IDF and TextRank, the algorithm extracts top keywords or tags from each post. These keywords are used to construct a tag network, and the tag

similarity is determined using the kernel diffusion method [22]. These similarity measures are used to calculate user similarity, and users are grouped using the Agglomerative Hierarchical clustering algorithm.

3.1. Extracting Tags from Social Media text

For tag extraction, preprocessing techniques like stop word removal, tokenization, and lemmatization are applied from social media text. Most-liked comments and re-comments are also considered to represent the post and identify individuals with similar opinions. The comment section plays a significant role in grouping individuals with similar thoughts into communities.

3.2. TF-IDF based Ranking

TF-IDF is used to rank keywords based on their importance in the document. It considers both term frequency and inverse document frequency, providing a reliable measure of relevance. For post representation, comments and re-comments are combined with the post's textual content and passed through the TF-IDF ranking algorithm to extract keywords and their scores. These results are saved for future use.

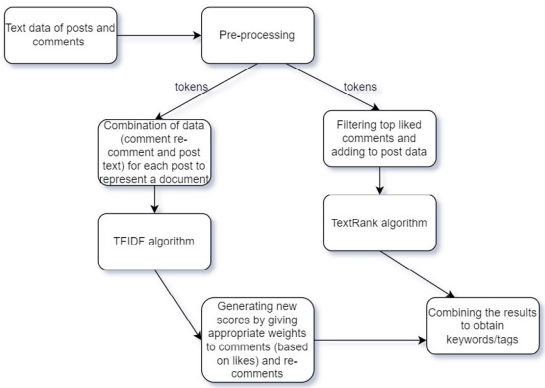


Fig. 1: Flow for generating tags

3.3. TextRank Algorithm

TextRank [23], similar to Google's PageRank, ranks text content using graph analysis. In TextRank, words represent nodes in a graph, and the edges capture word dependency relations, similar to a dependency tree [24]. The node's significance in the graph identifies important keywords to represent the posts. The importance of a node is calculated based on its in-degree and out-degree, normalized by the maximum degree. PageRank calculates the likelihood of moving between web pages, while TextRank applies a similar concept to words in a sentence. Both algorithms use an iterative process to determine probabilities, with TextRank determining sentence weights by identifying shared words. A graph-based approach is impractical for all comments and replies, so only well-liked comments represent posts. A criterion is applied to select significant comments: those with at least one like and at least one-fifth of the total post likes. This approach ensures that only important comments are considered, minimizing the risk of missing important comments for less popular posts or having excessively well-liked comments for popular posts. The selected comments and post contents are then subjected to the TextRank algorithm, providing the desired results for subsequent phases.

3.4. Mixed Ranking

Figure 1 summarizes the process we have followed to extract tags/keywords from the post, comment, and re-comment text data. TextRank and TF-IDF have their strengths and weaknesses. TextRank focuses on a single sentence for key-

word generation, while TF-IDF considers all documents. This can result in different outcomes. TextRank employs a graph-based approach to consider word relationships, ensuring comprehensive coverage. After text preprocessing, potential keywords (tokens) are generated. The final score for each token is determined by combining TF-IDF and TextRank scores, which are normalized on a scale of 0 to 1. The combined ranking algorithm 1 illustrates the contribution of TF-IDF scores to the final scores.

Algorithm 1 Combined Ranking algorithm

Input: Map/Dictionary mapping the word to score: FS, TFS, Array of words: Post, Comment
Output: Map/Dictionary converting the word to final score

procedure CombineScores($TFS, FS, Post, Comment$)
  for *tkn* in *Post* **do**
    FS_{tkn} += TFS_{tkn}
  end for
  for *tkn* in *Comment* **do**
    if *tkn* in *FS* **then**
      FS_{tkn} += $max(M_{tkn}/(NC + TLC)), 1/3) * TFS_{tkn}$
    else
      FS_{token} += $max(M_{tkn}/(NC + TLC)), 1/2) * TFS_{tkn}$
    end if
  end for
  return *FS*
end procedure

In the combined ranking algorithm, we calculate the final scores (FS_{token}) for each token (*tkn*) using the TF-IDF scores (TFS_{tkn}) and considering the number of comments (NC) and total likes on comments (TLC). Initially, the token's score is set to the TextRank algorithm's score. The multiplier value (M_{token}) for each token is calculated based on the number of comments it appears in and the total likes on those comments. This ensures that tokens appearing in posts have higher importance than those appearing only in the comments.

The values $\frac{1}{2}$ and $\frac{1}{3}$ are chosen through experimentation. Re-comments' scores are also considered, but with less significance. These scores are normalized between 0 and 1 using TF-IDF rankings and assigned appropriate weights for comments and re-comments. The resulting scores are added to the TextRank algorithm's scores. Typically, the number of single-word keywords ranges from 5 to 15, and the number of multi-word keywords ranges from 2 to 10, depending on the post size.

3.5. Grouping users based on the Tags they use

Now that the appropriate tags are extracted for each post, the formed tags are further used to find the similarity measures between users.

3.5.1. Tag network and Kernel Diffusion

Moving forward, instead of using the textual content, the posts are represented by the tags generated. To compare two posts, we can utilize the extracted tags and apply measures such as cosine similarity or count the number of common tags. However, these methods face a significant limitation as they assume that all users employ similar words in their posts, which may not be the case. Additionally, these methods overlook the significance of tags that are closely related but not identical. Therefore, it is crucial to employ an appropriate approach that establishes relationships between tags based on their usage in social media. In this regard, we utilize the tag network. The underlying concept behind the tag network is that if two keywords are consistently employed by users across social media, there is a high likelihood that they are semantically related to each other. To illustrate, let's consider the following tweets from the Twitter dataset:

1. *"Now all Apple has to do is get swype on the iphone and it will be crack. Iphone that is"*
2. *"Apple will be adding more carrier support to the iPhone 4S (just announced)"*
3. *"Hilarious youtube video - guy does a duet with apple 's Siri. Pretty much sums up the love affair! http://t.co/8ExbnQjY"*
4. *"RIM you made it too easy for me to switch to Apple iPhone. See ya!"*

Following are the keywords extracted from the tweets. (in the same order)

1. apple, iphone, swype, crack
2. apple, carrier, iphone
3. youtube, apple, siri, video, love
4. apple, iphone, RIM, switch

From the above tags, *apple* and *iphone* are used frequently. Therefore, they are considered to be semantically related and given more weight in the tag network than the pair of tags like *apple* and *youtube*. The tag network is defined in such a way that each node represents a tag and the edges between the node represent the frequency of users who used both tags collectively. The tag network can be formed by the algorithm 2. To prevent the impact of spamming

Algorithm 2 Tag network formation

Input:list of posts that are created by the user with user id as userid][tags$_{postid}$: list of tags for each post]
Output: 2D array W representing tag network in the adjacency matrix representation

```
procedure FormTagNetwork(userposts, tags)
    InitializeEmptyMatrix(W)
    for userid in userposts do
        edges := []
        for post in userposts_userid do
            for tag_1 in tags_post do
                for tag_2 in tags_post do
                    if (tag_1, tag_2) in edges then
                        continue
                    end if
                    append(edges, ((tag_1, tag_2))
                end for
            end for
        end for
        for ((tag_1, tag_2) in edges) do
            W_{tag_1 tag_2} += 1
        end for
    end for
    return W
end procedure
```

users on social media, we increment the weight for a pair of tags by one, representing the number of users using that specific pair of tags, rather than the number of posts. This approach maintains content relevance while minimizing spam influence. However, considering each post, the number of tag pairs to consider can become quadratic, so we limit the tags considered for representing posts.After constructing the tag network, we need to determine a suitable measure to assess tag similarity based on the graph structure. The shortest (weighted) distance between nodes is a common measure, but it may not be ideal here due to its sensitivity to edge changes and its limited consideration of edges. Computing the average distance of all paths is not feasible due to computational costs. Instead, we use the random walk method, which approximates the average distances between all paths connecting two nodes. This method, combined with kernel diffusion, offers a promising approach for measuring tag similarity [22]. Let's consider a tag network denoted as $W \in R^{t \times t}$, where t represents the total number of unique tags in all the posts. In this context, matrix L is defined according to the following formula,

$$L = W - D \tag{1}$$

where L is also known as the negated Laplacian matrix and D is a diagonal matrix and i_{th} diagonal element represents sum of all the elements in column i of matrix W representing the summation of all the edge weights adjacent to node i. The kernel K_β can be given by:

$$K_\beta = e^{\beta L} \tag{2}$$

where $\beta \geq 0$ is the diffusion parameter for controlling the speed of diffusion (no diffusion when $\beta = 0$). The details for calculation beta are discussed later in this section. The exponential (e^X) of a matrix is given by formula:

$$e^X = \sum_{k=0}^{\infty} \frac{1}{k!} X^k \tag{3}$$

As the Laplacian matrix L is a symmetric matrix, it can be subjected to Eigen decomposition given by:

$$L = V\Sigma V^T \tag{4}$$

where the columns of V are the eigenvectors and, Σ is a diagonal matrix whose diagonal elements are the eigenvalues. From above, the formula can be rewritten as:

$$K_\beta = Ve^{\beta\Sigma}V^T \tag{5}$$

Hence, the similarity measure between the tags can be given by $2D$ matrix K_β. From this point, the kernel is denoted by the symbol K. The more the calculated kernel value, the closer the tags. After calculating the value of kernel K, the similarity between two users is given by the formula

$$sim(u_i, u_j) = \sum_{t_1 \in tags(u_i)} \sum_{t_2 \in tags(u_j)} K(t_1, t_2) \frac{u_i(t_1)}{\sqrt{||u_i||}} \frac{u_j(t_2)}{\sqrt{||u_j||}} \tag{6}$$

where $sim(u_i, u_j)$ denotes the similarity measure between users (more is the measure more are the users similar), $tags(u_i)$ denotes the set of tags associated with user u_i, $u_i(t_1)$ denotes the number of times the tag t_1 is used by the user u_i and $|u_i|$ equals the number of times a user u_i used any tag. The key idea for the equation 6 is that the number of times a user used tags is also considered along with the similarity measure between tags used by the users. The usage of similarity measure between users $sim(u_i, u_j)$. The value of β is calculated by computing kernel K for different values of β and comparing the results against the cosine similarity method. The Twitter dataset was taken where each tweet contains the textual content by people on Twitter. Instead of taking the top tags, all the tags along with their scores are considered as the length of the tweets is relatively smaller. The cosine similarity measure between two tweets $cosinesim(tw_i, tw_j)$ is calculated by the formula:

$$cosinesim(tw_i, tw_j) = \sum_{t_1 \in tags(tw_i)} \sum_{t_2 \in tags(tw_j)} \frac{tagscore(tw_i, t_1)}{\sqrt{||tagscore(tw_i)||}} \frac{tagscore(tw_j, t_2)}{\sqrt{||tagscore(tw_j)||}} \tag{7}$$

where $tags(tw_i)$ is the set of tags representing tweet tw_i, $tagscore(tw_i, t_1)$ represents the score for the key word t_1 corresponding to tweet tw_i as discussed in the section 3.1 and $|tagscore(tw_i)|$ is the summation of scores of all the tags for tweet tw_i. The similarity measure between two tweets $tweetsim(tw_i, tw_j)$ calculated using Kernel diffusion is given by:

$$tweetsim(tw_i, tw_j) = \sum_{t_1 \in tags(tw_i)} \sum_{t_2 \in tags(tw_j)} K(t_1, t_2) \frac{1}{\sqrt{||tags(tw_i)||}} \frac{1}{\sqrt{||tags(tw_j)||}} \tag{8}$$

Here, $|tags(tw_i)|$ indicates the number of tags for the tweet tw_i. Equation 8 is quite similar to equation 6 the reason being that both the equations are based on kernel diffusion but equation 6 deals with the similarity between users and the latter deals with the similarity between the tweets. The numerator is 1 because a tag can only appear once in a tweet tw_i. Using equation 8, the top 200 similar tweets were extracted for all the tweets. The mean similarity value for the i_{th} similar post ($MS(i)$) is given by:

$$MS(i) = \frac{1}{n} \sum_{j=1}^{n} cosinesim(tw_j, sim_i(tw_j)) \tag{9}$$

where $sim_i(tw_j)$ indicates the i_{th} similar tweet similar to the tweet tw_j. The value $MS(i)$ is nothing but the average of the cosine similarity of the i_th similar tweet for every tweet. If the condition $MS(i) < MS(i-1)$ satisfies, this implies that the measure is good. Cosine similarity is considered the baseline comparison method as it is the most frequently used method for establishing the correlation between two vectors and the extracted keyword scores for the tweets can be represented as vectors. Figure 2 show the Mean cosine similarity values for the Kernel diffusion method with different values of β. The value of β is set to 10^{-5} as the line corresponds to $\beta = 10^{-5}$ is above all the other lines. It is notable that having a beta value less than 0.01 does not drastically affect the result. It is worth mentioning that the

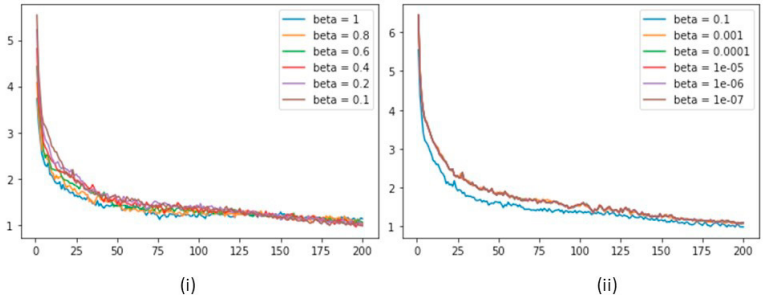


Fig. 2: (i) Mean similarity values for β from 0.1 to 1 (ii) Mean similarity values for β from 10^{-7} to 0.1

cosine similarity method mentioned above cannot be applied to the dataset when posts have much more keywords/tags as the comparison between two posts or two users can be computationally expensive.

3.5.2. Agglomerative Hierarchical Clustering

Nextly we focus on a clustering algorithm suitable for grouping users on social media while controlling the cluster size. The K-means algorithm is unsuitable for our data, which consists of similarity measures between users, not vectors. Instead, we use the K-medoids algorithm, where the medoid represents the center of the cluster by minimizing the average distance to all other nodes within the cluster, allowing for effective cluster size control.

K-medoids initially assigns randomly selected nodes to each cluster and iteratively updates the medoids based on a cost function to reduce the average distance between nodes within each cluster. However, a fixed value of k (the number of clusters) is not ideal; thus, we opt for Agglomerative Hierarchical clustering, where each node starts in its own cluster and similar clusters are merged until a threshold is met or all nodes belong to a single cluster. This method is successfully used in Community Detection, Social Influence Analysis, and User Segmentation. Various measures [25], [26] can be used to represent the similarity between two clusters, such as finding the distance between the closest or farthest pair of nodes. However, these measures have limitations. Instead, we consider the average distance between all pairs of nodes in the clusters. The algorithm 3 outlines the steps for Agglomerative Hierarchical clustering, efficiently finding the closest pair using the heap data structure to reduce the time complexity to $O(n^2 \log_2 n^2)$. The threshold limits the maximum number of users in a group to 50, but it can be adjusted as needed. Let's consider we have two clusters C_1 and C_2. Two of the measures that can be considered for finding the distance between two clusters C_1 and C_2 is :

$$distance(C_1, C_2) = min(sim(u_1, u_2)) \forall u_1 \in C_1, u2 \in C_2 \tag{10}$$

where u_1 is a user from the cluster C_1, u_2 is a user from cluster C_2, *min* denotes the minimum and *max* denotes the maximum functions. Equation 10 calculates the distance between two clusters as the distance between their closest pair of nodes. However, these measures are not ideal for large datasets as a single outlier node can significantly influence the result. To address this, we use another measure, the average distance between all pairs of nodes in two clusters, as shown in equation 11.

$$sim(C_1, C_2) = \frac{1}{||C_1||} \frac{1}{||C_2||} \sum_{u_1 \in C_1} \sum_{u2 \in C_2} sim(u_1, u_2) \tag{11}$$

where $||C_1||$ represents the number of users in cluster C_1.
The Agglomerative Hierarchical clustering is a seven steps approach namely

1. Compute pairwise similarity of all the nodes
2. Create similarity matrix from step 1
3. Initialize each node into a single cluster

4. Compute the distance between each pair of clusters
5. Construct the closeness matrix
6. Merge the closest pair of clusters
7. If the threshold is not met, go to step 4

The Agglomerative Hierarchical clustering is applied to a dataset of 9 nodes, ensuring a maximum cluster size of 4. A similar threshold of 50 users per group is used, which can be adjusted as needed. In step 4, computing the average distance between each pair of clusters can have a time complexity of $O(n^2)$, where n is the number of nodes. To optimize step 4, we avoid recalculating the similarity measure for each pair of clusters. The closeness matrix for each pair of clusters initially matches the original similarity matrix. After merging two clusters, only the similarity values between other clusters and the merged cluster change. Each cluster has a head node representing it, and when two clusters merge, the larger cluster's head node represents the merged cluster, and the head nodes for the nodes in the smaller cluster are updated accordingly. Let's consider the clusters C_1 and C_2 to be merged and let the new (merged) cluster be C_m, then the distance between the new cluster and any clusters (C_i) is given by the formula:

$$sim(C_m, C_i) = \frac{sim(C_i, C_1).\|C_1\| + sim(C_i, C_2).\|C_2\|}{\|C_1\| + \|C_2\|} \forall C_i \in clusters, C_i \notin \{C_1, C_2\} \tag{12}$$

Using equation 12, the time complexity for recalculating the similarity matrix and merging is $O(n)$. But finding the closest pair of clusters is still quadratic. Here we use the heap data structure. A heap is a data structure where numbers can be inserted and every time, the minimum element in the heap can be extracted in $O(log_2 n)$ time. Heap also supports the insert and remove operations with the same time complexity making it the best suitable data structure that can be used here. Therefore, every time we are recalculating the similarity matrix after the merge operation, the values $sim(C_1, C_i)$ and $sim(C_2, C_i)$ can be removed from the heap and $sim(C_m, C_i)$ is inserted. So, the total number of operations performed in the worst case for every merge is $3nlog_2 n^2$ which can be done n times. So, the reduced time complexity for the Hierarchical clustering algorithm is $O(n^2 log_2 n^2)$. The algorithm 3 illustrates the Agglomerative clustering.

Algorithm 3 Agglomerative Clustering

Input: users (list of users to be clustered), sim (2D matrix indicating the similarity between users), threshold (maximum cluster size and the minimum similarity required to be merged)
Output: Desired number of Clusters

```
procedure PERFORM-CLUSTERING(users, sim, threshold)
    initializEemptyHeap(heap)
    clusterSimilarity ← sim
    for user1 in users do
        for user2 in users do
            push(heap, clusterSimilarity[user1][user2])
        end for
    end for
    while not empty(heap) do
        C1, C2 = top(heap)
        pop(heap)
        if size(C1) + size(C2) > threshold1 or clusterSimilarity(C1, C2) < threshold2 then
            continue
        end if
        Cm = merge(C1, C2)
        recalculateClusterSimilarity(C1, C2, Cm)
        updateHeap(heap, C1, C2, Cm)
    end while
    return Clusters
end procedure
```

4. Experimentation and Results

Two datasets, *Facebook and Twitter*, are used to validate the proposed approach. The Facebook dataset contains 20,000 posts and 140,000 comments, while the Twitter dataset comprises over 5000 tweets. The algorithm's accuracy cannot be determined for the Facebook dataset since the users are not pre-grouped. Still, it serves as a suitable dataset for the problem. The Twitter dataset was chosen for evaluation and determining the value of β. Applying the algorithm to the Facebook dataset extracted 22000 keywords for post tags, combining TF-IDF and PageRank results. Figures 3 show the distribution of average scores for extracted hashtags using TF-IDF, PageRank, and mixed ranking. Around 35% of the hashtags overlap between TF-IDF and PageRank results, and approximately 89% of the posts have at

		Mediod	AHC
Cosine similarity	K=4	0.39	0.60
	K=10	0.42	0.59
	K=50	0.3	0.57
Kernel difusion	K=4	0.45	0.54
	K=10	0.50	0.54
	K=50	0.23	0.52

Table 1: F-scores for K-medoids algorithm and Agglomerative Hierarchical clustering algorithm on Twitter dataset

least one common hashtag. About 64% of the keywords from TF-IDF and 80% from PageRank are present in the final set of hashtags. The extracted keywords and tags are used to create a tag network for grouping individuals with similar interests. The same process was applied to the Twitter dataset, resulting in over 5000 tags. The K-medoids and hierarchical clustering algorithms were applied, and F-scores were obtained using different similarity measures, as shown in Tables 1. K-medoids perform better with the kernel diffusion-based similarity measure, while the opposite is observed for the agglomerative hierarchical clustering algorithm. Figure 4 visually represents aggregative hierarchical clustering applied to the Twitter dataset, classifying tweets into 4 clusters based on contextual similarities. The other figure shows a similar visualization for the Facebook dataset, clustering approximately 2,600 users without ground-truth information. The threshold of 50 users is the maximum cluster size.

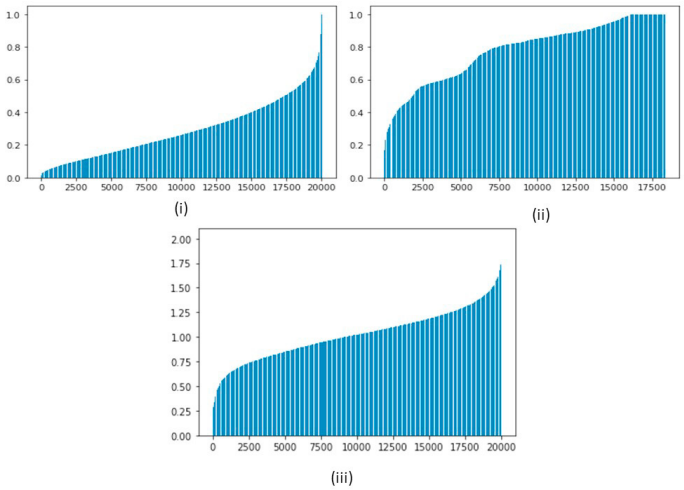


Fig. 3: (i) Average scores distribution for TF-IDF ranking (ii) Average scores distribution for TextRank algorithm (iii) Average scores distribution after combining the obtained results

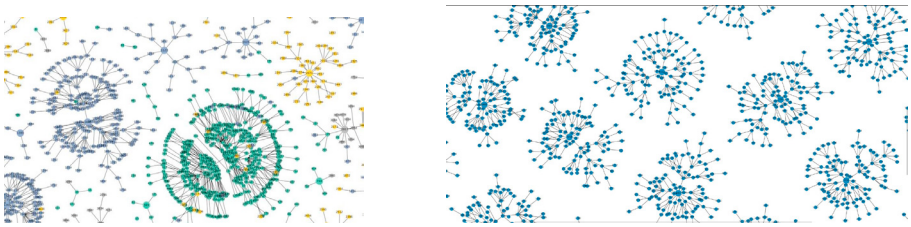


Fig. 4: Visualization of clustering on Twitter & Facebook

5. Conclusion and Discussion

We developed a methodical system for categorizing individuals by analyzing their social media texts, efficiently sorting users into specific clusters. Our initial testing on Twitter involved classifying tweets into four categories based on context. We then scaled up our approach to the extensive Facebook dataset, where we successfully formed clusters with up to 50 members per group, indicating users with similar interests. These positive outcomes suggest that our system is poised for smooth integration into various social media platforms. Our system can assist in the creation of interest-based groups when necessary.

References

[1] *Kaplan A, Mazurek G. Social media. Handbook of media management and economics, 273-86, 2018*

[2] *Davis, Robert, Inna Piven, and Michael Breazeale. "Conceptualizing the brand in social media community: The five sources model." Journal of Retailing and consumer services 21.4, 468-481, 2014*

[3] *Wellman, Barry. "The network community: An introduction." Networks in the global village. Routledge, 1-47, 2018*

[4] *Fortunato, Santo. "Community detection in graphs." Physics reports 486.3-5, 75-174, 2010*

[5] *Bedi, Punam, and Chhavi Sharma. "Community detection in social networks." Wiley interdisciplinary reviews: Data mining and knowledge discovery 6.3, 115-135, 2016*

[6] *Kanavos, Andreas, et al. "Emotional community detection in social networks." Computers & Electrical Engineering 65, 449-460, 2018*

[7] *Camacho D, Panizo-LLedot A, Bello-Orgaz G, Gonzalez-Pardo A, Cambria E. The four dimensions of social network analysis: An overview of research methods, applications, and software tools. Information Fusion, 63:88-120, 2020*

[8] *Ramasubbareddy S, Srinivas TA, Govinda K, Manivannan SS. Comparative study of clustering techniques in market segmentation. Innovations in Computer Science and Engineering: Proceedings of 7th ICICSE. 2020:117-25.*

[9] *Cai Z, Yang X, Huang T, Zhu W. A new similarity combining reconstruction coefficient with pairwise distance for agglomerative clustering. Information Sciences, 508:173-82, 2020*

[10] *Wang, Shuai, et al. "Identifying search keywords for finding relevant social media posts." Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 30. No. 1. 2016.*

[11] *Kovalchuk, Mikhail, and Denis Nasonov. "Hashtags: an essential aspect of topic modeling of city events through social media." 2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA). IEEE, 2021.*

[12] *Brooks, Christopher H., and Nancy Montanez. "Improved annotation of the blogosphere via autotagging and hierarchical clustering." Proceedings of the 15th international conference on World Wide Web. 2006.*

[13] *Xiao, Feng, Tomoya Noro, and Takehiro Tokuda. "News-Topic Oriented Hashtag Recommendation in Twitter Based on Characteristic Co-occurrence Word Detection." ICWE. 2012.*

[14] *Otsuka, Eriko, Scott A. Wallace, and David Chiu. "Design and evaluation of a twitter hashtag recommendation system." Proceedings of the 18th International Database Engineering & Applications Symposium. 2014.*

[15] *Sivasangari, V., et al. "Isolating rumors using sentiment analysis." Journal of Cyber Security and Mobility (2018): 181-200.*

[16] *Wang, Xufei. Connecting users with similar interests for group understanding. Arizona State University, 2013.*

[17] *C. R. Athira and N. M. Dhanya, "Three dimensional geo-tweet visualization system for spatio-temporal events," 2017 Second International Conference on Electrical, Computer and Communication Technologies (ICECCT), Coimbatore, India, pp. 1-6, 2017*

[18] *L. S. Nair and S. M. K, "Knowledge Graph based Question Answering System for Remote School Education," 2022 International Conference on Connected Systems & Intelligence (CSI), Trivandrum, India, 2022, pp. 1-5, doi: 10.1109/CSI54720.2022.9924128.*

[19] *Nair, Lekshmi S., M. K. Shivani, and Jo Cheriyan. "Enabling Remote School Education using Knowledge Graphs and Deep Learning Techniques." Procedia Computer Science 215, 618-625., 2022*

[20] *Y. P. Kommineni, S. Somanchi, B. S. Pavan Kumar, Y. Ladani and L. S. Nair, "Social Media Text to Formal Representation: A Knowledge Graph Approach," 2023 9th International Conference on Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, pp. 1691-1696, 2023*

[21] *Jaffali, Soufiene, et al. "Clustering and classification of like-minded people from their tweets." 2014*

[22] *Risi, Imre Kondor. "Diffusion kernels on graphs and other discrete input spaces." Proc. 19th Int. Conf. Machine Learning, 2002. 2002.*

[23] *Page, Lawrence, et al. The PageRank citation ranking: Bringing order to the web. Stanford InfoLab, 1999.*

[24] *Mihalcea, Rada, and Paul Tarau. "Textrank: Bringing order into text." Proceedings of the 2004 conference on empirical methods in natural language processing. 2004.*

[25] *Nair, Lekshmi S., Swaminathan Jayaraman, and Sai Pavan Krishna Nagam. "An improved link prediction approach for directed complex networks using stochastic block modeling." Big Data and Cognitive Computing 7.1 (2023): 31.*

[26] *Jishag, A. C., et al. "Automated review analyzing system using sentiment analysis." Ambient Communications and Computer Systems: RACCCS-2018. Springer Singapore, 2019.*